DAY 1 OF 3 · INSTRUCTOR-LED TRAINING

Introduction to Terraform on Google Cloud

Day 1: Terraform foundations and secure provisioning

Prepared for Jack Henry & Associates · 09:00–17:00

TODAY'S GUIDING QUESTION

How do we turn a small Google Cloud deployment into a repeatable and reviewable Terraform project?

DAY OUTCOME

A working, parameterized deployment with useful outputs, a verified GCS backend, and a documented approach to identities and secrets.

Day 1 agenda

Time	Module	Topic
09:00-09:30	**M01**	Infrastructure as code and the course scenario
09:30-10:30	**M02**	Project structure, providers, and authentication
10:30-10:45	Break	
10:45-12:00	**M03**	Resources and the Terraform command workflow
12:00-13:00	Lunch	
13:00-14:45	**M04**	Variables, data sources, outputs, and files
14:45-15:00	Break	
15:00-16:15	**M05**	State and Google Cloud Storage backends
16:15-17:00	**M06**	Secrets and least privilege

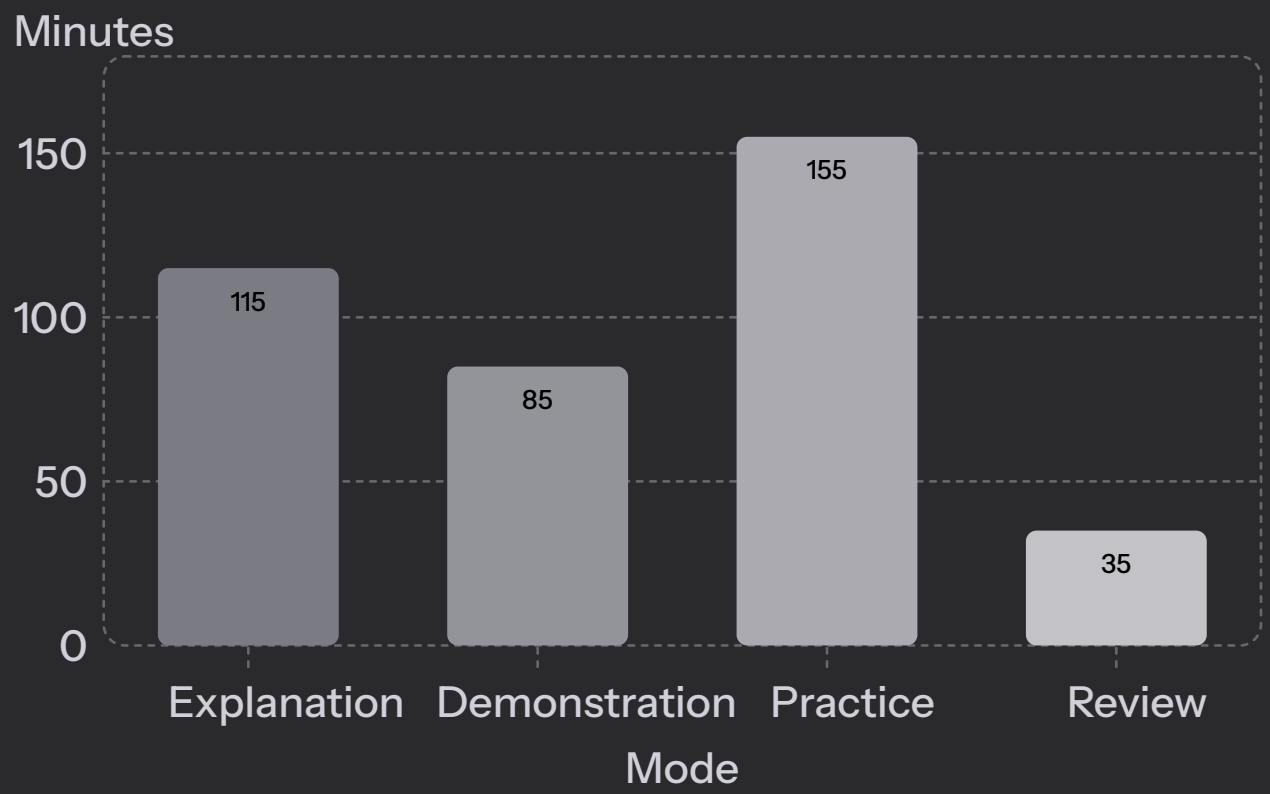
Who this course is for and how the day runs

Who it is for

DevOps engineers and technical managers with cloud fundamentals, Google Cloud console, projects, core services, and basic Git. No prior Terraform or Packer experience assumed.

Where Day 1 fits

Day 1 builds a working deployment and trustworthy state. Day 2 extends these with abstraction and controlled change.



390 instructional minutes, by mode

THREE-DAY COURSE

Course learning outcomes across all three days

01 **Plan and apply**

Build a structured project through a reviewed plan and apply workflow.

02 **Reuse**

Use variables, data sources, outputs, functions, collections, meta-arguments, and modules for reuse.

03 **State**

Configure a GCS backend; explain state, locking, version history, recovery, and environment boundaries.

04 **Identity and secrets**

Apply least-privilege identity and secret handling across code, state, saved plans, and logs.

05 **Operations**

Detect drift, import resources, diagnose failures, and explain landing zones and Packer handoffs.

M01

MODULE 01 · 09:00-09:30

Infrastructure as code and the course scenario

Why we declare infrastructure, how Terraform tracks it, and the environment we build across the course.

M01 outcome: configuration, state, and resources are three different things

01 Explain the relationship among configuration, state, and deployed resources

02 Distinguish infrastructure provisioning from software configuration

TIME BUDGET

30 min

- Explain 15 min
- Demo 5 min
- Practice 5 min
- Review 5 min

M01 · INFRASTRUCTURE AS CODE

Infrastructure as code: declare the intent, let the tool converge

Declarative intent

Describe the desired end state; Terraform works out the actions.

Repeatability

The same configuration produces the same environment.

Source control

Every change is a reviewable commit, supporting enterprise change management.

M01 · CORE MODEL

Configuration, state, and real resources

WHAT YOU WRITE

Configuration

.tf files describing desired infrastructure

plan →

WHAT TERRAFORM RECORDS

State

Terraform's record mapping resource addresses to remote objects

apply →

WHAT EXISTS

Resources

The real objects in Google Cloud

Editing a file alone changes nothing in the cloud; only a reviewed plan and apply does.

M01 · SCOPE

Provisioning infrastructure is not configuring software

Terraform provisioning

Networks · Subnets · Instances · IAM · Buckets

INSIDE THE INSTANCE

Software configuration

Packages and settings inside the instance

- Prepared machine images
- Startup configuration
- Ongoing configuration management

The change sequence every team follows

Request

The need is raised and tracked.

Code change

Edit the `.tf` files.

Plan

Show the proposed actions.

Review

A second person reads the plan.

Apply

Apply the reviewed plan.

Verification

Check the resources exist.

Recorded evidence

Keep proof of what changed.

The course scenario: a repeatable environment for a small Linux application

Who

A fictional internal service team; a teaching example, not Jack Henry's actual architecture.

Starting point

A VPC, a subnet, narrowly scoped connectivity, and a Compute Engine instance.

Across the course

Parameterize, move state to GCS, extract a module, review a second-region expansion, reconcile an existing resource.

Growth

Multi-team ownership grows from a few instances to a regional expansion.

D01 demo and pair activity

D01 · DEMO

Follow a small configuration change through a prepared plan to a deployed resource, and see where state records the association.

PAIR ACTIVITY

Pairs classify six actions, such as creating a subnet, installing packages, and approving a plan, by responsible tool or team.

CHECK

Explain why changing a configuration file alone does not change an existing cloud resource.

M02

MODULE 02 · 09:30–10:30

Project structure, providers, and authentication

Lay out the repository, pin the toolchain, and prove which identity and project Terraform will use.

M02 · OUTCOMES

M02 outcome: a pinned, initialized project running as the right identity

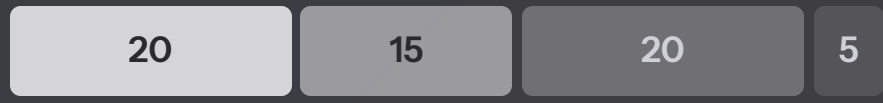
01 Navigate a Terraform project

02 Initialize a pinned provider configuration

03 Verify the identity and project the lab uses

TIME BUDGET

60 min



- Explain 20 min
- Demo 15 min
- Practice 20 min
- Review 5 min

A conventional root configuration layout

File	Holds
`main.tf`	Resources
`providers.tf`	Provider configuration
`versions.tf`	`required_version` and `required_providers`
`variables.tf` and `outputs.tf`	The module's interface
`README`	How to use it

Terraform loads all .tf files in a module together; filenames are organizational conventions.

Pinning Terraform and the Google provider

- `required_version` constrains the Terraform CLI version; 1.11 or later gives you ephemeral values and write-only arguments used in M06
- `required_providers` declares the `hashicorp/google` source and a version constraint
- `~> 8.0` tracks the current major release (8.x) and blocks an unreviewed jump to 9.0
- The `provider` block sets explicit project, region, and zone
- `terraform init` records the selected provider version in `.terraform.lock.hcl`

```
terraform {
  required_version = ">= 1.11"

  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 8.0"
    }
  }
}

 "google" ">provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}
```

Version constraint operators

Operator	Meaning
`= 1.2.0`	Exactly this version (also the meaning of a bare version)
`!=`	Exclude a version
`>` `>=` `<` `<=`	Comparisons
`~> 8.0`	Allows only the rightmost component to increment (8.x, not 9.0)
`~> 8.2.1`	Allows 8.2.x only (patch increments), not 8.3.0

Allowed vs selected

Constraints set what is allowed; the lock file records what was selected.

Upgrading

`terraform init -upgrade` selects newer versions within the constraints.

What goes in Git and what never does

COMMIT

- `.tf` files
- `README`
- `.terraform.lock.hcl`

EXCLUDE

- The `.terraform` cache directory
- Credentials
- State files and saved plans

⚠ State and plan files can contain sensitive values in plaintext.

M02 · AUTHENTICATION

Authenticating with Application Default Credentials

ADC is Google's recommended way to authenticate Terraform to Google Cloud.
Workstation setup: `gcloud auth application-default login`

ADC search order

1. The `GOOGLE_APPLICATION_CREDENTIALS` environment variable
2. The gcloud ADC file
3. The attached service account, from the metadata server

Use service account impersonation where prepared.

M02 · AUTHENTICATION

Verify the identity and project before you run anything

Command	Confirms
`gcloud auth list`	The active gcloud account
`gcloud config list`	Active account, project, and default region/zone
`gcloud auth application-default print-access-token`	ADC can mint a token

Set project, region, and zone explicitly in the provider rather than relying on defaults.

L01 records the intended project and identity as evidence.

Service account keys versus Workload Identity Federation

LONG-LIVED

Service account keys

Powerful long-lived credentials.

A security risk if mismanaged.

FEDERATED

Workload Identity Federation

External workloads use federated identities instead of keys.

GITHUB GITLAB AWS AZURE

ANY OIDC OR SAML 2.0 IDP

WIF grants access directly or through service account impersonation.

Service account impersonation from a workstation

Your user credentials request short-lived tokens for a service account; no key file is created.

Required role

The caller needs Service Account Token Creator (`roles/iam.serviceAccountTokenCreator`) on that service account.

Use only where the lab has prepared it.

Provider argument

```
provider "google" {
  impersonate_service_account = "SA_EMAIL"
}
```

gcloud alternative

```
gcloud auth application-default login \
  --impersonate-service-account=SA_EMAIL
```

Three identities, three different jobs

Backend access

Reads and writes state in the GCS bucket.

Provider access

Calls Google Cloud APIs to manage resources.

Workload service account

Attached to the running instance for the application's own access.

M02 · DEMO AND LAB

D02 demo and L01 start

D02 · DEMO

Inspect the starter files, run initialization, and trace a prepared federation flow from an external identity to Google Cloud access.

L01 · LAB

Tool and version checks, verify lab identity and project, inspect the repository, initialize successfully.

Use prepared credentials; do not create long-lived keys.

EVIDENCE

Intended project and identity recorded; approved provider installed with no secrets in configuration.

M03

MODULE 03 · 10:45–12:00

Resources and the Terraform command workflow

Write the starter resources, run the command workflow, and read a plan before anything changes.

M03 outcome: deploy, verify, and read every plan action

01 Create and verify a small Google Cloud deployment

02 Interpret proposed create, update, replace, and destroy actions

TIME BUDGET

75 min

- Explain 20 min
- Demo 15 min
- Practice 35 min
- Review 5 min

Anatomy of a resource block

```
resource "google_compute_network" "main" {
  name                    = "app-network"
  auto_create_subnetworks = false
}
```

Exported attribute:
`google_compute_network.main.id`

Resource type

`google_compute_network`

Local label

The name used inside the module.

Address

`type.label`, such as `google_compute_network.main`

Arguments and attributes

Arguments you set; attributes Terraform exports after creation.

References

A reference to another resource's attribute creates an implicit dependency that orders operations.

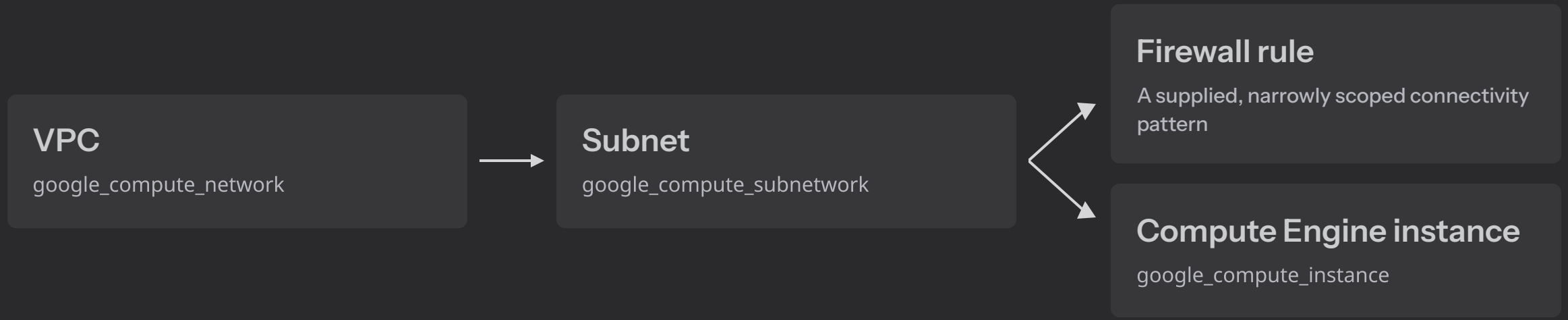
Example: network, subnet, and instance in HCL

```
resource "google_compute_network" "main" {
  name = "app-network"
  auto_create_subnetworks = false
}
resource "google_compute_subnetwork" "app" {
  name = "app-subnet"
  network = google_compute_network.main.id
  ip_cidr_range = "10.10.0.0/24"
  region = var.region
}
```

```
resource "google_compute_instance" "app" {
  name = "app-vm"
  machine_type = "e2-small"
  zone = var.zone
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-13"
    }
  }
  network_interface {
    subnetwork = google_compute_subnetwork.app.id
  }
}
```

Implicit dependencies: subnet references network; instance references subnet. Names and sizes are illustrative.

The starter environment: VPC, subnet, connectivity, instance



The Google provider is the interface to Google Cloud APIs.

The subnet references the network; the instance references the subnet.

The core command workflow

01 `fmt`

Canonical formatting

02 `init`

Install providers, configure backend

03 `validate`

Check configuration syntax and consistency

04 `plan`

Evaluate the proposed infrastructure change

05 `apply`

Make the change

06 `show` **and** `output`

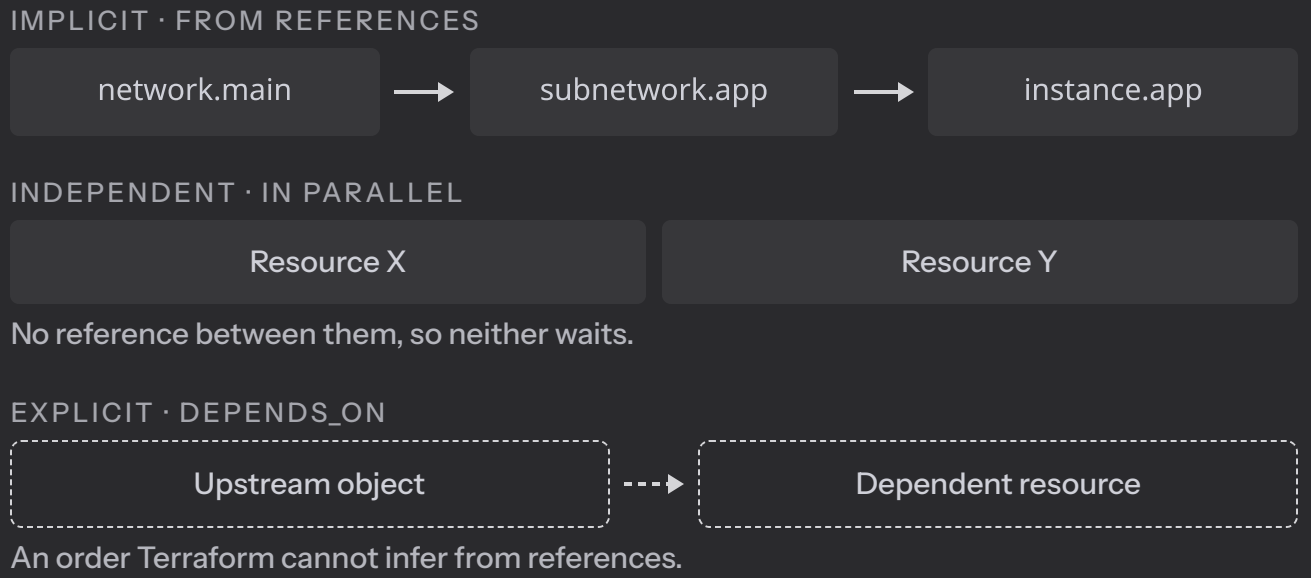
Inspect state, plans, and outputs

07 `destroy`

Remove managed resources

Implicit and explicit dependencies

- Terraform builds a dependency graph from references between blocks
- Independent resources are created in parallel; dependent ones wait
- `depends_on` declares a dependency Terraform cannot infer; use it sparingly
- `terraform graph` outputs the graph in DOT format for visualization
- **D03** shows how one referenced value changes the graph



Reading a plan before you apply

Symbol	Action
`+`	Create
`~`	Update in place
`-/+`	Replace
`-`	Destroy

Read the summary

The summary line counts add, change, and destroy.

Apply what was reviewed

Save a plan with `-out` and apply exactly that reviewed plan.

Verify afterwards

Check in Google Cloud and run a second plan: it should propose no changes.

D03 demo and L01 continued

D03 · DEMO

Provision the starter environment, annotate addresses and the plan summary, and show one referenced value's effect on the dependency graph.

L01 · LAB

Complete resource arguments, run fmt and validate, review a saved plan, apply, check the instance in the console.

Destroy only a disposable resource; keep the main deployment for later modules.

EVIDENCE

Resources exist, outputs identify them, and an unchanged plan proposes no changes.

M04

MODULE 04 · 13:00–14:45

Variables, data sources, outputs, and files

Replace hard-coded values with inputs, read existing objects, and publish outputs others can use.

M04 · OUTCOMES

M04 outcome: parameterized, data-aware, and consumable

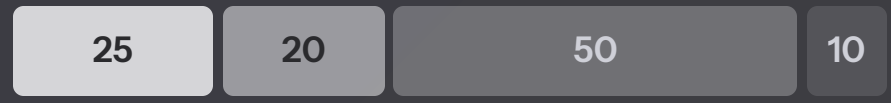
01 Parameterize the deployment

02 Query an existing Google Cloud object

03 Expose outputs people and simple automation can consume

TIME BUDGET

105 min



- Explain 25 min
- Demo 20 min
- Practice 50 min
- Review 10 min

Input variables

- Declare with `type`, `default`, and `description`
- `validation` blocks reject bad values at plan time with a clear error message
- Supply values with `.tfvars` files
- Prefer explicit types over `any`

```
variable "region" {
  type        = string
  description = "Region for the lab deployment"

  validation {
    condition     = contains(["us-central1", "us-east1"], var.region)
    error_message = "Region must be an approved lab region."
  }
}
```

Illustrative region list.

Type constraints

PRIMITIVE

`string`

`number`

`bool`

COLLECTION

`list(type)`

`set(type)`

`map(type)`

STRUCTURAL

`object({...})`

`tuple([...])`

ANY

`any`

Accepts any type; avoid where an explicit type is possible.

Explicit types catch mistakes at plan time and document the interface.

Variable precedence, lowest to highest

A later source overrides an earlier one.

01 Default in the declaration

02 Environment variables (`TF_VAR_name`)

03 `terraform.tfvars`

04 `terraform.tfvars.json`

05 `*.auto.tfvars` and `*.auto.tfvars.json`, in lexical order

06 `-var` and `-var-file` on the command line, in the order given

Data sources read; resources manage

A `data` block queries an existing object without managing it.

Example: `data "google_compute_image"` looks up an approved image by family and project.

Reference returned attributes, such as `data.google_compute_image.app.self_link`

```
data "google_compute_image" "app" {
  family  = var.image_family
  project = var.image_project
}

initialize_params {
  image = data.google_compute_image.app.self_link
}
```

Use `initialize_params` inside the instance's `boot_disk`.

Locals, references, interpolation, and outputs

`locals`

Name a computed value once and reuse it.

Interpolation

`"${var.env}-app"` builds strings.

`output` blocks

Expose identifiers like instance name and internal IP.

`terraform output -json`

Machine-readable values for a small inventory report.

M04 · FILES

Static files and startup templates

file(path)

Reads an existing file as a string; it does not create a dependency on a later file-generation step.

templatefile(path, vars)

Renders a template, such as a startup script, with supplied values.

path.module

Builds paths relative to the module's directory.

Configuration files are source; state and saved plans are runtime artifacts.

D04 demo and L01 completion

D04 · DEMO

Replace hard-coded settings with inputs, look up an image, render a startup template, extract non-secret outputs into an inventory.

L01 · LAB

Parameterize location and instance settings, add validation and an image lookup, produce useful outputs.

Final 30 minutes: solve the change request with progressively fewer hints.

EVIDENCE

Invalid value rejected, intended image used, correct inventory identifiers, clean unchanged plan.

M05

MODULE 05 · 15:00–16:15

State and Google Cloud Storage backends

Move state off a single machine into a locked, versioned Cloud Storage backend.

M05 · OUTCOMES

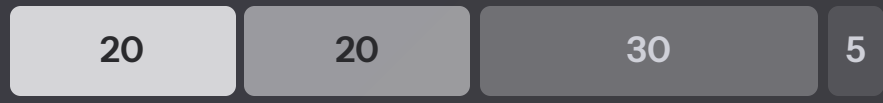
M05 outcome: team-ready state in GCS

01 Move lab state to its designated GCS backend

02 Explain how access, locking, and version history support team operation

TIME BUDGET

75 min



- Explain 20 min
- Demo 20 min
- Practice 30 min
- Review 5 min

What state is and how to inspect it

State associates resource addresses with remote objects.

By default state is a local `terraform.tfstate` file, which is hard for teams sharing infrastructure.

`terraform state list`

Every tracked address

`terraform state show ADDRESS`

The recorded attributes for one address

M05 · STATE

Local versus remote state

LOCAL

One machine's view, no shared locking.

REMOTE

Shared, locked, access-controlled.

- Separate state per project or per lab
- Provider authentication and backend authentication are configured and permissioned separately

Backend configuration rules

- One `backend` block inside the `terraform` block per configuration
- Backend blocks cannot reference variables, locals, or data sources
- Partial configuration: supply values at init with `-backend-config=KEY=VALUE` or a file
- Changing backend settings requires re-running `terraform init`
- Give each learner or pair a distinct prefix

The GCS backend

- Stores state as an object under a configurable prefix in a pre-existing bucket; the bucket must exist first
- Supports state locking
- HashiCorp strongly recommends Object Versioning for recovery from accidental deletion and human error
- Restrict bucket access; bootstrap the backend separately from the configuration that uses it

```
terraform {
  backend "gcs" {
    bucket = "BUCKET_NAME"
    prefix = "labs/PAIR_ID"
  }
}
```

Placeholder values; use your assigned bucket and prefix.

M05 · LOCKING

How state locking protects concurrent runs

01

Acquire

Operations that could write state, such as `plan` and `apply`, acquire a lock first.

02

Contend

A second writer fails with a lock error showing who holds the lock.

03

Wait

`-lock-timeout=DURATION` makes Terraform wait for a lock to be released.

04

Recover

`terraform force-unlock LOCK_ID` is an exceptional recovery action, covered on Day 3.

M05 · MIGRATION

Migrating state safely

1

Declare

Add the `backend` block.

2

Migrate

Run `terraform init -migrate-state`.

3

Verify

Check the remote object and versioning.

4

Confirm

Same addresses with `state list` and a clean plan.

5

Retain

Keep the original checkpoint under controlled access until migration is confirmed.

D05 demo and L02 lab

D05 · DEMO

Backend bootstrap, migrate a disposable example, and show a competing writer hitting a lock.

L02 · LAB

Migrate your deployment into your assigned bucket and prefix, verify the object and versioning, confirm addresses and a clean plan.

Use protected machine-readable state or plan output for audits.

EVIDENCE

Correct location, unchanged addresses, and why a shared prefix risks collision.

M06

MODULE 06 · 16:15–17:00

Secrets and least privilege

Find where secrets leak and give each identity only the access its job needs.

M06 · OUTCOMES

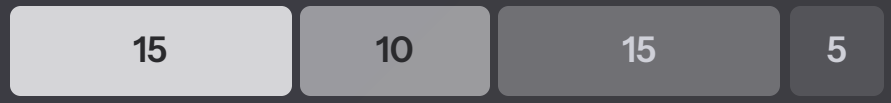
M06 outcome: close the exposure paths, narrow the permissions

01 Identify secret-exposure paths

02 Assign appropriate permissions to the Terraform runner, backend user, and application identity

TIME BUDGET

45 min



- Explain 15 min
- Demo 10 min
- Practice 15 min
- Review 5 min

M06 · SECRETS

Keep plaintext secrets out of code

No secrets in source or committed variable files.

HashiCorp Vault and Google Secret Manager as secret stores.

Prefer a runtime secret reference, where the workload reads the secret itself.

A secret manager alone does not keep state clean

Values persist

Secret values used in configuration are stored in state and plan files.

Redaction is not encryption

`sensitive = true` redacts display; it is not encryption.

Avoid persistence

Ephemeral values (Terraform 1.10+) and write-only arguments (1.11+) avoid persistence, where the provider supports them.

Protect the artifacts

State, plan files, JSON outputs, logs, and build artifacts.

M06 · LEAST PRIVILEGE

Least privilege and separation of duties

Deployment runner

Only the roles needed to manage its resources.

State storage

Bucket access limited to authorized runners and operators.

Runtime identity

Only what the application needs.

Automation

Prefer WIF over service account keys.

Example role assignments for the lab identities

Identity	Example roles	Scope
Runner (Terraform)	`roles/compute.networkAdmin`, `roles/compute.securityAdmin`, `roles/compute.instanceAdmin.v1`	The lab project
Runner (Terraform)	`roles/iam.serviceAccountUser`	The runtime service account only
Backend user	`roles/storage.objectAdmin`	The state bucket only
Runtime service account	`roles/secretmanager.secretAccessor`	One secret the app needs

⊗ Grant at the narrowest resource level available; avoid basic roles like Owner and Editor. Note which roles are needed only because of what the runner creates: `securityAdmin` exists here only for the firewall rule, and `serviceAccountUser` only so the runner can attach the runtime identity to the VM.

M06 · DEMO AND LAB

D06 demo and L03 review

D06 · DEMO

A deliberately fake secret shows source exposure, output redaction, and state persistence, then a safer runtime-reference design and a Vault/GSM comparison.

L03 · LAB

Remove an embedded secret, restrict one excessive grant, and identify artifacts needing controlled access.

EVIDENCE

Explain why a secret manager alone does not guarantee a secret-free state file; identify separate runner and runtime permissions.

DAY 1 WRAP-UP

Day 1 wrap-up and retrieval check

1. What will this plan create, update, replace, or destroy?
2. How do inputs, data sources, and outputs connect?
3. Which state and backend prefix are selected?
4. Where could a secret or excessive permission remain?

Tomorrow: functions, meta-arguments, modules, workspaces, drift, and import.